

Project Report

Topic: Loan Eligibility Prediction

1. Introduction

Loan eligibility prediction models are designed to optimize the decision-making process, reduce risk, and foster trust between lenders and borrowers.

1.1 Background:

i. High Volume of Applications

Financial institutions process thousands of loan applications daily. Manually assessing each application is time-consuming and prone to errors.

ii. Risk of Default

Lending money inherently carries the risk of default, where borrowers fail to repay their loans. Identifying applicants with a high likelihood of repayment is crucial to protect the institution's financial health.

iii. Bias and Inconsistencies

Human decision-making can introduce biases and inconsistencies, leading to unfair practices and potential regulatory non-compliance.

iv. Time Pressure

Customers demand quick loan approvals, especially in competitive markets. A lengthy decision-making process can lead to customer dissatisfaction or loss of business.

v. Role of Technology and Data

Advances in machine learning and data analytics offer solutions to these challenges. By leveraging historical data on previous loans (e.g., income, employment status, credit score, loan amount, repayment history), predictive models can evaluate the eligibility of new applicants with high accuracy and speed.

1.2 Motivation:

i. Streamlining the Loan Approval Process

- **Automation:** Loan eligibility prediction automates the initial screening of loan applications, significantly reducing manual effort and processing time.
- **Efficiency:** It enables quick decisions, allowing lenders to process a large volume of applications without delays.

ii. Reducing Credit Risk

- **Minimizing Defaults:** By accurately predicting a borrower's ability to repay, lenders can minimize the risk of granting loans to individuals who are likely to default.
- **Data-Driven Decisions:** Models consider multiple factors like income, credit history, and existing liabilities, improving the accuracy of risk assessments.

iii. Enhancing Customer Experience

- **Faster Approvals:** Quick decisions enhance the customer experience, building trust and satisfaction with the financial institution.
- **Personalization:** Predictive models can also suggest tailored loan products based on a customer's financial profile.

iv. Improving Fairness and Transparency

- **Objective Assessment:** Predictive models reduce human biases in loan approval decisions by using consistent criteria based on historical data.
- **Regulatory Compliance:** They help institutions ensure compliance with lending laws by providing documented, data-driven decision-making processes.

v. Cost Efficiency

- **Operational Savings:** By automating a significant part of the loan application review, institutions can lower operational costs and allocate resources more effectively.
- **Fraud Detection:** Models can help identify fraudulent applications, further reducing financial losses.

vi. Support for Financial Inclusion

- **Extending Credit Access:** Predictive models can help assess loan eligibility for underrepresented groups by analyzing non-traditional data sources, enabling more people to access credit.
- **Innovative Products:** They pave the way for innovative financial products like microloans and credit for small businesses, fostering economic growth.

1.3 Objectives:

The objective behind loan eligibility prediction is to determine whether an applicant qualifies for a loan based on specific criteria and historical data. This predictive model helps financial institutions like banks and lending companies make informed, consistent, and efficient decisions while minimizing risk.

2. Literature Review

2.1 Existing Research on Loan Eligibility Prediction

- **Historical Studies:**

- ◆ Discuss early research papers that explored statistical methods for loan default risk or credit scoring (e.g., logistic regression and decision trees).
- ◆ Highlight their limitations in handling large-scale, non-linear data.

- **Modern Machine Learning Approaches:**

- ◆ Outline the rise of machine learning techniques like Random Forest, Gradient Boosting, and Neural Networks in solving loan prediction tasks.
- ◆ Mention studies leveraging publicly available datasets such as LendingClub or Kaggle datasets for loan-related predictions.

- **Feature Engineering:**

- ◆ Summarize research on key features (e.g., income, employment status, credit history, and debt-to-income ratio) and their impact on model performance.
- ◆ Discuss the role of domain knowledge in feature selection and transformation.

2.2 Methodologies Used in Loan Prediction

Various machine learning algorithms have been used to address classification problems, each offering unique strengths depending on the complexity and nature of the data. This section discusses several common methodologies used to solve classification tasks.

i. Logistic Regression

Logistic Regression is a linear model used for binary classification. It estimates the probability of a class belonging to a given data point, outputting a value between 0 and 1.

- **Advantages:**

- Simple and interpretable.
- Works well when the relationship between features and the target is linear.
- Provides probabilities, useful in decision-making.

- **Disadvantages:**

- Struggles with non-linear relationships.
- Assumes independence of features.

ii. K-Nearest Neighbors (KNN)

KNN is a non-parametric classifier that assigns a class to a point based on the majority class of its k nearest neighbors.

Advantages:

- Simple and easy to understand.
- Effective for small, structured datasets.
- No training phase required (lazy learning).

Disadvantages:

- Computationally expensive for large datasets (due to distance calculations).
- Sensitive to the choice of k and irrelevant features.
- Requires feature scaling.

iii. Support Vector Machine (SVM)

SVM is a powerful classifier that works by finding a hyperplane that best separates classes. It can handle both linear and non-linear classification through the use of kernel functions.

Advantages:

- Effective in high-dimensional spaces.
- Can handle non-linear decision boundaries with kernels.
- Robust to overfitting, especially in high-dimensional data.

Disadvantages:

- Computationally expensive for large datasets.
- Sensitive to choice of kernel and hyperparameters.
- Less interpretable than linear models.

iv. Decision Tree

A Decision Tree splits data into subsets based on feature values, creating a tree-like model of decisions. Each leaf node represents a class label.

Advantages:

- Easy to interpret and visualize.
- Handles both categorical and numerical data.
- No need for feature scaling.

Disadvantages:

- Prone to overfitting, especially with deep trees.
- Instability: small changes in data can result in different splits.

v. Random Forest

Random Forest is an ensemble of decision trees trained on bootstrapped samples of the data, with predictions averaged across trees.

Advantages:

- Reduces overfitting by averaging multiple decision trees.
- Handles large datasets well and is robust to noise.
- Can provide feature importance.

Disadvantages:

- Less interpretable compared to a single decision tree.
- Computationally expensive, especially with many trees.

vi. Gradient Boosting

Gradient Boosting builds models sequentially, with each new model correcting the errors of the previous ones. Popular implementations include XGBoost and LightGBM.

Advantages:

- Often provides high predictive accuracy.
- Can handle complex datasets and non-linear relationships.
- Less prone to overfitting than individual decision trees.

Disadvantages:

- Computationally expensive and slower to train.
- Requires careful hyperparameter tuning.
- More prone to overfitting if not tuned properly.

Conclusion :

Logistic Regression is best for simple, linear problems with interpretability needs. **KNN** is useful for small datasets but is computationally expensive for large datasets. **SVM** works well in high-dimensional spaces but requires tuning and can be slow. **Decision Trees** are interpretable but prone to overfitting. **Random Forest** mitigates overfitting and is robust, though less interpretable. **Gradient Boosting** provides high accuracy but is computationally intensive and requires careful tuning.

3. Dataset Description

3.1 Source: Kaggle

3.2 Features:

The following are key features used in loan eligibility prediction:

- Number of Dependents
- Education
- Self-Employed Status
- Annual Income (Income Annum)
- Loan Amount
- Loan Term
- CIBIL Score
- Residential Assets Value
- Commercial Assets Value
- Luxury Assets Value
- Bank Asset Value

3.3 Target Variable:

- Loan Status

4. Data Preprocessing

4.1 Load and Read the Dataset

- **Reading dataset:** We use the following code for reads a CSV (Comma-Separated Values) file into a pandas DataFrame.

```
1 data = pd.read_csv("/content/loan_approval_dataset.csv")
```

The resulting DataFrame that stores the contents of the CSV file in a tabular structure in data.

Then we use the code to see first five row of the dataset

```
1 data.head(5)
```

```
[6] data.head(5)
```

	loan_id	no_of_dependents	education	self_employed	income_annum	loan_amount	loan_term	cibil_score	residential_assets_value	commercial_assets_value	luxury_assets_value	bank_asset_value	loan_status
0	1	2	Graduate	No	9600000	29900000	12	778	2400000	17600000	22700000	8000000	Approved
1	2	0	Not Graduate	Yes	4100000	12200000	8	417	2700000	2200000	8800000	3300000	Rejected
2	3	3	Graduate	No	9100000	29700000	20	506	7100000	4500000	33300000	12800000	Rejected
3	4	3	Graduate	No	8200000	30700000	8	467	18200000	3300000	23300000	7900000	Rejected
4	5	5	Not Graduate	Yes	9800000	24200000	20	382	12400000	8200000	29400000	5000000	Rejected

Figure 1: Example figure

It use for quickly inspecting the structure, column names, and a sample of the data. To know about the dimensions of the DataFrame data, we use

```
1 data.shape
```

Output:- (4269, 13)

To removing spaces from column names improves code readability, prevents errors in column access, and ensures compatibility with various tools and systems. So modifying the column names by removing any spaces (" ") and replacing them with an empty string("") with this code,

```
1 data.columns = data.columns.str.replace(" ", "")
```

and check the updated column names by the code,

```
1 data.columns
```

Output:- Index(['loan_id', 'no_of_dependents', 'education', 'self_employed', 'income_annum', 'loan_amount', 'loan_term', 'cibil_score', 'residential_assets_value', 'commercial_assets_value', 'luxury_assets_value', 'bank_asset_value', 'loan_status'], dtype='object')

4.2 Handling missing values and Outliers

- **For missing values:**

Checking for Missing Values: The code starts by checking if there are any missing values in the dataset using the `isnull()` method. It calculates the sum of missing values for each column.

```
1 data.isnull().sum()
```

This will output the number of missing values for each column. The output you provided indicates that there are no missing values in any of the columns (each shows 0 missing values).

Output:-

Column Name	Missing Values
loan_id	0
no_of_dependents	0
education	0
self_employed	0
income_annum	0
loan_amount	0
loan_term	0
cibil_score	0
residential_assets_value	0
commercial_assets_value	0
luxury_assets_value	0
bank_asset_value	0
loan_status	0

- **Removing Outliers & Infinite values using Z-Scores Method**

```
1      # Calculate Z-scores
2  z_scores = np.abs(stats.zscore(data[['no_of_dependents', '
3      income_annum', 'loan_amount', 'loan_term', 'cibil_score',
4      'residential_assets_value', 'commercial_assets_value', '
5      luxury_assets_value', 'bank_asset_value']]))
6
7  # Identify outliers where Z-score > 3
8  outliers = (z_scores > 3)
9
10 # Remove outliers
11 filtered_data_z = data[~outliers.any(axis=1)]
12 print(filtered_data_z)
```

Identify and remove rows with outliers from a dataset using Z-scores.

Steps

1. Calculate Z-Scores

- ◆ Use `scipy.stats.zscore` to compute Z-scores for numerical columns.
- ◆ Formula :

$$Z = \frac{X - \mu}{\sigma}$$

where:

- * X : Value
- * μ : Mean of the column
- * σ : Standard deviation of the column

2. Identify Outliers

- ◆ Rows with any $|Z| > 3$ are marked as outliers.

3. Remove Outliers

- ◆ Use `outliers.any(axis=1)` to identify rows with outliers.
- ◆ Filter the DataFrame to keep only rows without outliers using `~` (logical NOT operator).

Note:-

- ◆ **Z-Score Threshold:** Default is 3 but can be adjusted based on requirements.
- ◆ **Handling Multiple Outliers:** Rows with even one outlier are removed.
- ◆ **Scaling:** Ensure numerical data is consistently scaled before applying this method.

- **Identifying Duplicates**

```
1 # Checking for duplicated values
2 data.duplicated().sum()
```

Output:- 0

It shows that how many row that are identical to a previous row in the DataFrame.

4.3 Visualize Distribution of Features with Countplot:

In classification models, a *Countplot* is used to visualize the distribution of categorical features by showing the count of each category. It helps identify class imbalances, where certain categories may be overrepresented or underrepresented, which can impact model performance. Additionally, it provides insights into the frequency of each category, aiding in data preprocessing and feature engineering decisions.

Code:

```
1 # Visualize Distribution of "feature" Categories with Countplot
2 sns.countplot(x="feature", data= data, palette="deep")
3 plt.show()
```

4.4 Visualize Pairwise Relationships using Heatmap:

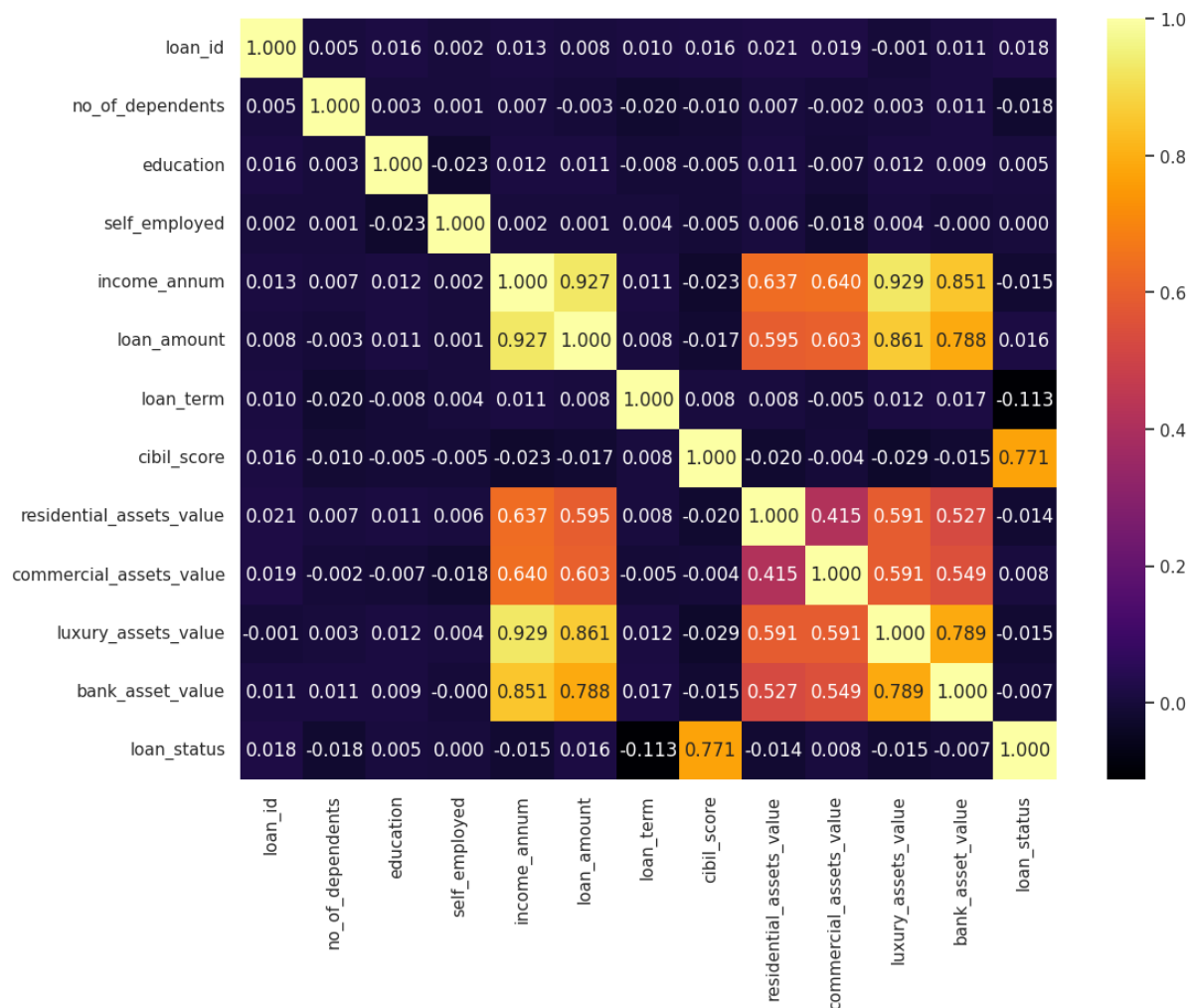
In classification models, a *heatmap* is used to visualize pairwise relationships between features. It represents the correlation matrix of numerical features, where color intensity shows the strength of correlation between pairs. This helps in:

- **Identifying Correlations:** Highlights which features are strongly correlated, aiding in feature selection and reducing multicollinearity.
- **Data Insights:** Reveals patterns, clusters, and relationships between variables, providing deeper insights into the data.
- **Feature Engineering:** Assists in choosing relevant features by showing redundant features with high correlation.

Code:

```
1 plt.figure(figsize=(12,9))
2 sns.heatmap(data.corr(), annot=True, cmap='coolwarm', fmt='.3f')
3 plt.show()
```

Output:-



4.5 Histogram Distribution

In classification models, to visualize the distribution of numerical features we used *Histogram Distribution*. This is important because:

- **Identify Patterns:** It helps in understanding the distribution of data for each class, which can reveal patterns or imbalances between classes.
- **Data Preprocessing:** A histogram can identify skewed distributions, which might need transformation (e.g., log transformation) to improve model performance.
- **Feature Selection:** It can help identify important features and outliers, aiding in feature engineering or selection.

Overall, histograms help assess the suitability of numerical features for classification, ensuring the model handles data effectively.

Code:

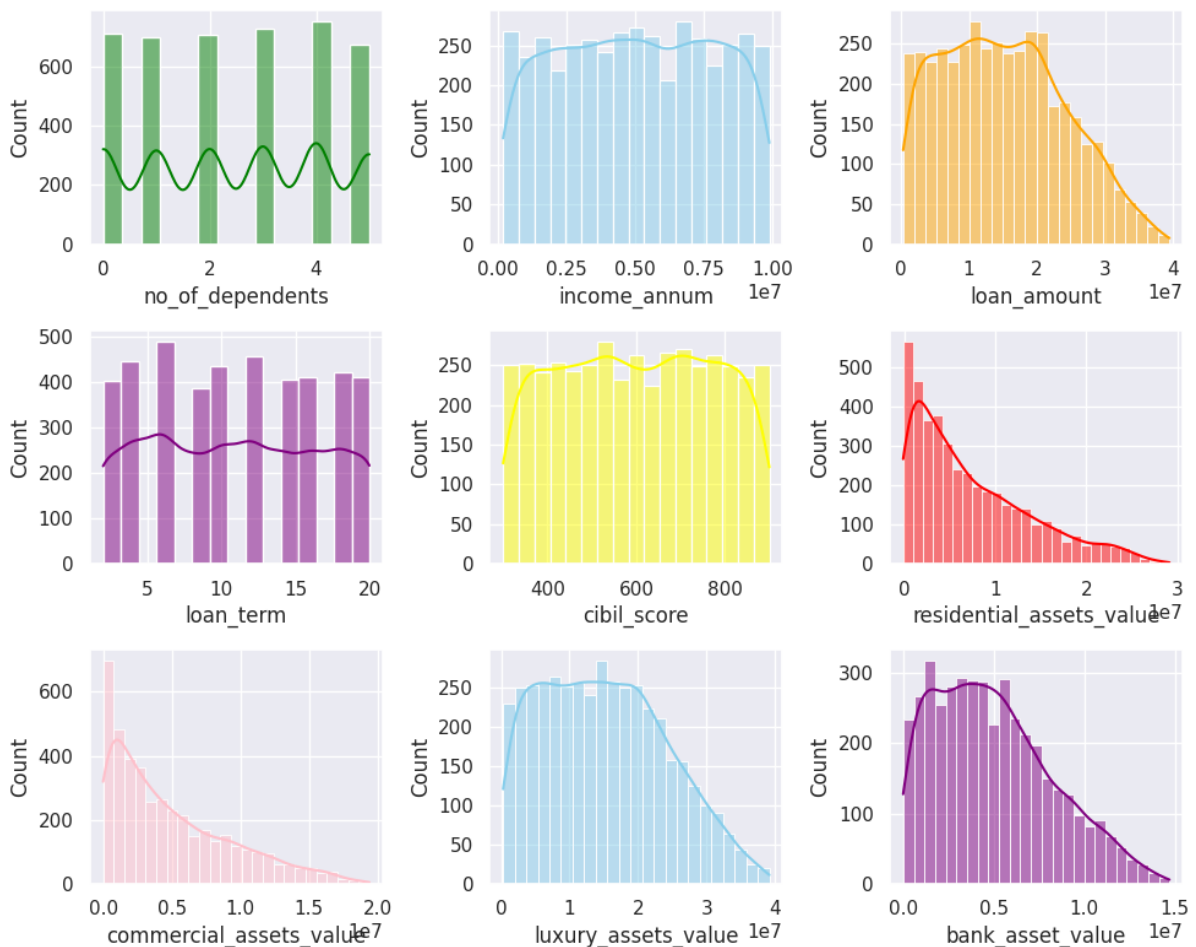
```
1 sns.set(style="darkgrid")
2 fig, axs = plt.subplots(3, 3, figsize=(10, 8))
3
```

```

4 sns.histplot(data=data, x="no_of_dependents", kde=True, ax=axes[0, 0],
  color='green')
5 sns.histplot(data=data, x="income_annum", kde=True, ax=axes[0, 1], color
  ='skyblue')
6 sns.histplot(data=data, x="loan_amount", kde=True, ax=axes[0, 2], color=
  'orange')
7 sns.histplot(data=data, x="loan_term", kde=True, ax=axes[1, 0], color='
  purple')
8 sns.histplot(data=data, x="cibil_score", kde=True, ax=axes[1, 1], color=
  'yellow')
9 sns.histplot(data=data, x="residential_assets_value", kde=True, ax=axes
  [1, 2], color='red')
10 sns.histplot(data=data, x="commercial_assets_value", kde=True, ax=axes
  [2, 0], color='pink')
11 sns.histplot(data=data, x="luxury_assets_value", kde=True, ax=axes[2,
  1], color='skyblue')
12 sns.histplot(data=data, x="bank_asset_value", kde=True, ax=axes[2, 2],
  color='purple')
13
14
15 plt.tight_layout()
16 plt.show()

```

Output:-



4.6 Feature Engineering

- **Encoding:**

In machine learning, feature encoding is crucial for converting categorical variables into numerical values, as most machine learning models require numerical input. Categorical features often contain string labels, which need to be transformed into numerical values to make the data compatible with these algorithms.

Below, we describe the encoding process for three categorical features in the dataset: `education`, `loan_status`, and `self_employed`. The goal is to replace the categorical values with numerical values, which will allow our machine learning models to interpret them effectively.

- ◆ **Encoding the education Feature**

The `education` feature contains two categories: *"Not Graduate"* and *"Graduate"*. We encode these categories as follows:

```
1 data['education'] = data['education'].replace({" Not Graduate":  
        0, " Graduate": 1})
```

Explanation: In this encoding step, we replace the "Not Graduate" category with the value 0 and "Graduate" with the value 1. This transformation makes the feature compatible with machine learning models, allowing them to process the data numerically.

- ◆ **Encoding the loan_status Feature**

The `loan_status` feature contains two categories: *"Approved"* and *"Rejected"*. We encode these categories as follows:

```
1 data['loan_status'] = data['loan_status'].replace({" Approved":  
        1, " Rejected": 0})
```

Explanation: In this step, we replace the "Approved" category with the value 1 and the "Rejected" category with the value 0. This transformation is important because most machine learning algorithms cannot work with string values like "Approved" or "Rejected," and instead, they need numerical representations.

- ◆ **Encoding the self_employed Feature**

The `self_employed` feature contains two categories: *"Yes"* and *"No"*. We encode these categories as follows:

```
1 data['self_employed'] = data['self_employed'].replace({" No":  
        0, " Yes": 1})
```

Explanation: Here, we replace the category "No" with the value 0 and "Yes" with the value 1. As with the other encoding steps, this transformation is necessary to convert the data into a numerical format that can be used in machine learning models.

Why We Transform the Data?

These transformations are essential because machine learning models cannot process raw categorical data (such as strings). By encoding the categorical features into numerical values, we ensure that the data is in a format that can be efficiently used by the models. Moreover, encoding helps the model identify relationships between the features and the target variable, which improves model performance.

- **Drop Unnecessary Variables:**

In our dataset a feature called 'loan_id' is not important for prediction. So we drop the feature from data set.

Code:

```
1 data = data.drop(columns=['loan_id'])
```

Why We Drop Unnecessary Variables?

Dropping unnecessary variables is an essential step in data preprocessing because it helps reduce the complexity of the model and improves computational efficiency. Irrelevant or redundant features can introduce noise, leading to overfitting and decreased model performance. By removing such variables, we ensure that the model focuses only on the most important features, enhancing accuracy and interpretability.

- **Data Normalization:**

Data normalization is the process of scaling numerical features to a common range, usually between 0 and 1, using techniques like Min-Max scaling. This ensures that all features contribute equally to the model, preventing features with larger numerical values from dominating the learning process.

Formula:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

where:

- x is the original value.
- $x_{\min}$ is the minimum value in the feature.
- $x_{\max}$ is the maximum value in the feature.

```
1 scaling = MinMaxScaler()  
2 X_train_normalize = scaling.fit_transform(X_train)  
3 X_test_normalize = scaling.transform(X_test)
```

Why We can do Data Normalization?

Data normalization is the process of scaling features to a common range, typically $[0, 1]$ or $[-1, 1]$, to ensure that all features contribute equally to the model. It is crucial for models like K-Nearest Neighbors, Support Vector Machines, and Neural Networks, which are sensitive to the scale of the features. Normalization helps prevent features with larger scales from dominating the model and speeds up convergence in gradient-based algorithms like Logistic Regression. However, tree-based models like Decision Trees, Random Forests, and Gradient Boosting do not require normalization, as they are not affected by the scale of the features.

At last our data is prepared.

5. Methodology

5.1 Algorithms Used:

The following algorithms are used in the project

- **Logistic Regression**
- **K-Nearest Neighbour (KNN)**
- **Support Vector Machine (SVM)**
- **Decision Tree**
- **Random Forest**
- **Gradient Boosting**

5.2 Justification:

i. Logistic Regression

Why Suitable:

- Logistic Regression is a linear model used for binary classification problems.
- It predicts the probability of the target variable belonging to a class (e.g., loan approved or rejected).
- Its simplicity and interoperability make it ideal for understanding which features impact the approval decision.

Strengths:

- Works well with small to medium-sized datasets.
- Can handle linearly separable data effectively.

Limitations:

- Struggles with complex, non-linear relationships.

ii. K-Nearest Neighbors (KNN)

Why Suitable:

- KNN is a non-parametric algorithm that classifies data based on the similarity (or distance) between data points.
- It works well when the relationship between features and the target variable is non-linear.

Strengths:

- Easy to implement and understand.
- No assumptions about data distribution.

Limitations:

- Computationally expensive for large datasets.
- Sensitive to irrelevant or unscaled features.

iii. Support Vector Machine (SVM)

Why Suitable:

- SVM is effective in high-dimensional spaces and works well for binary classification tasks.
- It can find an optimal hyperplane that maximizes the margin between two classes.

Strengths:

- Effective with non-linear data using kernels (e.g., radial basis function).
- Robust to overfitting in high-dimensional data.

Limitations:

- Computationally expensive for large datasets.
- Hyperparameter tuning (e.g., kernel choice, regularization) can be complex.

iv. Decision Tree

Why Suitable:

- Decision Trees are rule-based models that divide the dataset into smaller subsets based on feature values.
- They are highly interpretable and can handle both linear and non-linear relationships.

Strengths:

- Handles categorical and numerical data well.
- Easy to visualize and explain decisions.

Limitations:

- Prone to overfitting, especially with deep trees.

v. Random Forest

Why Suitable:

- Random Forest is an ensemble learning technique based on multiple decision trees, combining their outputs for better predictions.
- It reduces overfitting and improves generalization.

Strengths:

- Robust to noise and overfitting.
- Handles large datasets and missing values effectively.

Limitations:

- Can be computationally intensive.
- Less interpretable compared to single decision trees.

vi. Gradient Boosting (e.g., XGBoost, LightGBM)

Why Suitable:

- Gradient Boosting builds models sequentially, with each model correcting errors made by the previous one.
- It is highly effective for imbalanced or complex datasets.

Strengths:

- High prediction accuracy.
- Works well with non-linear relationships.
- Customizable to handle imbalanced data (e.g., tuning class weights).

Limitations:

- Computationally expensive, especially for large datasets.
- Sensitive to hyperparameter tuning.

General Suitability for Loan Prediction

1. **Binary Classification:** All these models are designed for binary classification, making them naturally suitable for predicting loan approval/rejection.
2. **Feature-Target Relationship:** They can handle a mix of linear and non-linear relationships between borrower features (e.g., income, credit score) and the target variable.
3. **Imbalanced Datasets:** Models like Random Forest and Gradient Boosting can handle imbalanced datasets, which is common in loan prediction problems (e.g., fewer loans rejected than approved).
4. **Interpretability:** Logistic Regression, Decision Trees, and Random Forest provide insights into which features are important, helping stakeholders understand the decisions.

6. Implementation

6.1 Tools and Libraries

- **NumPy**: For numerical operations and working with arrays and matrices.

Code:

```
1 import numpy as np
```

- **Pandas**: For data manipulation, cleaning, and preprocessing.

Code:

```
1 import pandas as pd
```

- **Matplotlib**: For data visualization, including plotting charts and graphs.

Code:

```
1 import matplotlib.pyplot as plt
```

- **Seaborn**: For statistical data visualization and creating more complex plots like heatmaps and pair plots.

Code:

```
1 import seaborn as sns
```

- **Time**: For measuring execution time and performing time-based operations.

Code:

```
1 import time
```

- **Scipy(stats)**: A module in the SciPy library that provides a large number of statistical functions, including methods for computing Z-scores, probability distributions, statistical tests, and more.

Code:

```
1 from scipy import stats
```

- **Scikit-learn (preprocessing)**: For data preprocessing, such as scaling and transforming features.

Code:

```
1 from sklearn.preprocessing import MinMaxScaler
```

- **Scikit-learn (model selection)**: For splitting data into training and testing sets, and for hyperparameter tuning.

Code:

```
1 from sklearn.model_selection import train_test_split
```

- **Scikit-learn (metrics)**: For model evaluation metrics such as accuracy, precision score, recall score, f1 score and confusion matrix.

Code:

```
1 from sklearn.metrics import accuracy_score, precision_score,
  recall_score, f1_score, confusion_matrix
```

- **Scikit-learn (tree)**: For decision tree models and related functions.

Code:

```
1 from sklearn import tree
2 from sklearn.tree import plot_tree
```

- **LogisticRegression**: For building logistic regression models.

Code:

```
1 from sklearn.linear_model import LogisticRegression
```

- **KNeighborsClassifier**: For implementing the K-Nearest Neighbors classification algorithm.

Code:

```
1 from sklearn.neighbors import KNeighborsClassifier
```

- **SVC (Support Vector Classifier)**: For building Support Vector Machine classification models.

Code:

```
1 from sklearn.svm import SVC
```

- **DecisionTreeClassifier**: For implementing decision tree-based classification models.

Code:

```
1 from sklearn.tree import DecisionTreeClassifier
```

- **RandomForestClassifier**: For implementing random forest classification models.

Code:

```
1 from sklearn.ensemble import RandomForestClassifier
```

- **GradientBoostingClassifier**: For building gradient boosting classification models.

Code:

```
1 from sklearn.ensemble import GradientBoostingClassifier
```

- **RandomizedSearchCV**: For hyperparameter tuning and searching for the best model configuration.

Code:

```
1 from sklearn.model_selection import RandomizedSearchCV
```

- **GridSearchCV**: For hyperparameter tuning and searching for the best model configuration.

Code:

```
1 from sklearn.model_selection import GridSearchCV
```

6.2 Parameters

In this section, we discuss important hyperparameters for the classification models used in the loan approval prediction problem.

- **Random Forest Classifier:**

- **n_estimators**: Number of trees in the forest. Default: 100.
Increasing the number of trees can improve performance but at the cost of computation time.
- **max_depth**: Maximum depth of each tree. Default: None.
Limiting the depth helps avoid overfitting, especially on noisy datasets.
- **min_samples_split**: Minimum number of samples required to split an internal node. Default: 2.
Increasing this can prevent the model from creating overcomplex trees and overfitting.
- **max_features**: The number of features to consider when looking for the best split. Default: 'auto' (sqrt(number of features)).
Restricting the number of features can help in reducing overfitting.

- **Logistic Regression:**

- **C**: Inverse of regularization strength. Smaller values specify stronger regularization. Default: 1.0.
Lower values of 'C' increase regularization, which can prevent overfitting, but might underfit if set too low.
- **penalty**: The regularization term ('l1', 'l2', or 'elasticnet'). Default: 'l2'.
L1 can lead to sparse solutions (some coefficients are exactly zero), while L2 tends to shrink coefficients toward zero.

- **solver**: The algorithm to use in the optimization problem ('liblinear', 'newton-cg', 'lbfgs', etc.). Default: 'lbfgs'.
Different solvers may perform better depending on the dataset size and model complexity.
- **max_iter**: Maximum number of iterations for the solver to converge. Default: 100.
Increasing this value can help the model converge in case of very large datasets.

- **Support Vector Classifier (SVC):**

- **C**: Regularization parameter. The strength of the regularization is inversely proportional to 'C'. Default: 1.0.
Higher values of 'C' can lead to overfitting, while lower values may underfit.
- **kernel**: Specifies the kernel type to use ('linear', 'rbf', 'poly', 'sigmoid'). Default: 'rbf'.
The 'rbf' kernel is often a good default, but linear or polynomial kernels may be more suitable for certain datasets.
- **gamma**: Kernel coefficient for 'rbf', 'poly', and 'sigmoid'. Default: 'scale'.
The 'gamma' value controls the influence of a single training example; small values can make the model more general, while large values may lead to overfitting.
- **degree**: Degree of the polynomial kernel function. Default: 3.
Only relevant if 'kernel='poly'; higher values can lead to more complex decision boundaries.

- **K-Nearest Neighbors (KNN):**

- **n_neighbors**: The number of neighbors to use for classification. Default: 5.
Choosing a smaller value makes the model more sensitive to noise, while a larger value might smooth over important distinctions.
- **weights**: Weight function used in prediction ('uniform', 'distance', or a callable). Default: 'uniform'.
'distance' gives more weight to closer neighbors, making the model more sensitive to local patterns.
- **algorithm**: Algorithm used to compute the nearest neighbors ('auto', 'ball_tree', 'kd_tree', 'brute'). Default: 'auto'.
Choosing the appropriate algorithm can speed up the computation, depending on the dataset size.
- **leaf_size**: Leaf size passed to BallTree or KDTree. Default: 30.
Smaller values can lead to faster searches, but increase memory usage.

- **Decision Tree Classifier:**

- **max_depth**: Maximum depth of the tree. Default: None.
Limiting depth can prevent overfitting by reducing the complexity of the model.

- **min_samples_split**: Minimum number of samples required to split an internal node. Default: 2.
Increasing this can help reduce overfitting by requiring more samples before splitting a node.
- **criterion**: Function to measure the quality of a split ('gini' for Gini impurity or 'entropy' for information gain). Default: 'gini'.
'entropy' might lead to better results in some cases but can be computationally expensive.
- **splitter**: Strategy used to split at each node ('best' or 'random'). Default: 'best'.
'random' can sometimes speed up training and provide similar results in large datasets.

- **RandomizedSearchCV:**

- **n_iter**: The number of iterations for random search. Default: 10.
This controls how many different combinations of hyperparameters will be tested.
- **param_distributions**: The distribution of hyperparameters to sample from. You can specify a range of values or distributions for each hyperparameter. For example, for 'n_estimators', you could try [100, 200, 300], and for 'max_depth', you might try [3, 5, 7].
- **scoring**: The scoring method to use for evaluating models.
You can use metrics such as accuracy, precision, recall, etc., depending on the goal of your model.
- **cv**: The number of cross-validation folds to use. Default: 5.
A higher number of folds can provide a more reliable estimate of model performance, but at the cost of computation time.
- **random_state**: Seed for random number generation. Default: None.
Setting this ensures reproducibility of the search results.

- **Gradient Boosting Classifier:**

- **n_estimators**: Number of boosting stages to be run. Default: 100.
Increasing this number can improve the model's performance, but also the risk of overfitting.
- **learning_rate**: Determines the impact of each tree on the final model. Default: 0.1.
Smaller values lead to slower learning but can result in better generalization.
- **max_depth**: Maximum depth of each individual tree. Default: 3.
Limiting the depth helps to prevent overfitting.
- **subsample**: The fraction of samples used to fit each individual tree. Default: 1.0.
Values less than 1.0 can make the model more robust by introducing randomness.

6.3 Training Process

- **Process:**

1. **Separate Features and Target Variable:**

- ◆ X contains all columns except `loan_status`, representing the input features.
- ◆ y contains the `loan_status` column, representing the target variable.

Code:

```
1 # Separate features and target variable
2 X = data.drop(['loan_status'], axis=1)
3 y = data['loan_status']
```

2. **Split the Dataset:**

- ◆ The data is split into training and testing sets using `train_test_split`.
- ◆ 30% of the data (`test_size=0.3`) is allocated for testing, and 70% is used for training.
- ◆ A fixed `random_state` ensures reproducibility of the split.

Code:

```
1 # Split the dataset into training and testing sets
2 X_train, X_test, y_train, y_test = train_test_split(
3     X, y, test_size=0.3, random_state=1)
```

3. **Validation:**

To validate the model, three distinct methods were employed:

- ◆ **GridSearchCV**: Exhaustively searches a predefined hyperparameter grid using cross-validation, tuning the model's hyperparameters to find the best combination.

Code:

```
1 # Define the hyperparameters grid
2 param_grid = { Hyperparameters
3                 ... }
4
5 # Initialize GridSearchCV
6 grid_search = GridSearchCV(estimator=model, param_grid=
7                             param_grid, cv=5, n_jobs=-1, verbose=1)
8
9 # Fit the model
10 grid_search.fit(X_train, y_train)
11
12 # Best hyperparameters
13 print("Best Hyperparameters:", grid_search.best_params_)
```

- ◆ **RandomizedSearchCV**: Randomly samples from a set of hyperparameters, reducing the search time while still optimizing model performance through cross-validation.

Code:

```

1 # Define the hyperparameters grid for random sampling
2 param_dist = { Hyperparameters
3                 ... }
4 # Initialize RandomizedSearchCV
5 random_search = RandomizedSearchCV(estimator=model,
6                                     param_distributions=param_dist,
7                                     n_iter=100, cv=5, n_jobs=-1, random_state
8                                     =42, verbose=1)
9
10 # Fit the model
11 random_search.fit(X_train, y_train)
12
13 # Best hyperparameters
14 print("Best Hyperparameters:", random_search.best_params_)

```

- ◆ **Splitting the Test Set:** The initial test set was further split into a validation set (50% of the test set) and a testing set. This allows tuning the model on the validation set and evaluating it on the final test set. Additionally, hyperparameters were fine-tuned by plotting performance graphs to observe how different parameter combinations impact model accuracy.

Code:

```

1 scoreListmodel = []
2 for i in range(m,n):
3     Model = model(hyperparameter = i)
4     Model.fit(X_train, y_train)
5     scoreListknn.append(Model.score(X_val_model,
6                                     y_val_model))
7
8 plt.plot(range(m,n), scoreListmodel)
9 plt.xticks(np.arange(m,n,1))
10 plt.xlabel("Hyperparameter value")
11 plt.ylabel("Score")
12 plt.show()

```

7. Result

In this section, we present the evaluation results for each model, which include accuracy, training time, and confusion matrix. The results are summarized in the table and graph below:

7.1 Accuracy Score Matrix

Model	Accuracy (%)
Decision Tree	98.75
Gradient Boosting	98.52
Random Forest	97.66
SVM	94.69
Logistic Regression	92.27
KNN	91.42

Table 1: Accuracy Comparison

7.2 Precision score Matrix

Model	Precision
Random Forest	0.990123
Gradient Boosting	0.985056
Decision Tree	0.983254
SVM	0.965561
KNN	0.954198
Logistic Regression	0.949807

Table 2: Precision Comparison

7.3 Recall score Matrix

Model	Recall
Decision Tree	0.997573
Gradient Boosting	0.991228
Random Forest	0.973301
SVM	0.948622
Logistic Regression	0.924812
KNN	0.910194

Table 3: Recall Comparison

7.4 F1_Score score Matrix

Model	F1 Score
Decision Tree	0.990361
Gradient Boosting	0.988132
Random Forest	0.981640
SVM	0.957016
Logistic Regression	0.937143
KNN	0.931677

Table 4: F1_Score Comparison

7.5 Computational Time Matrix

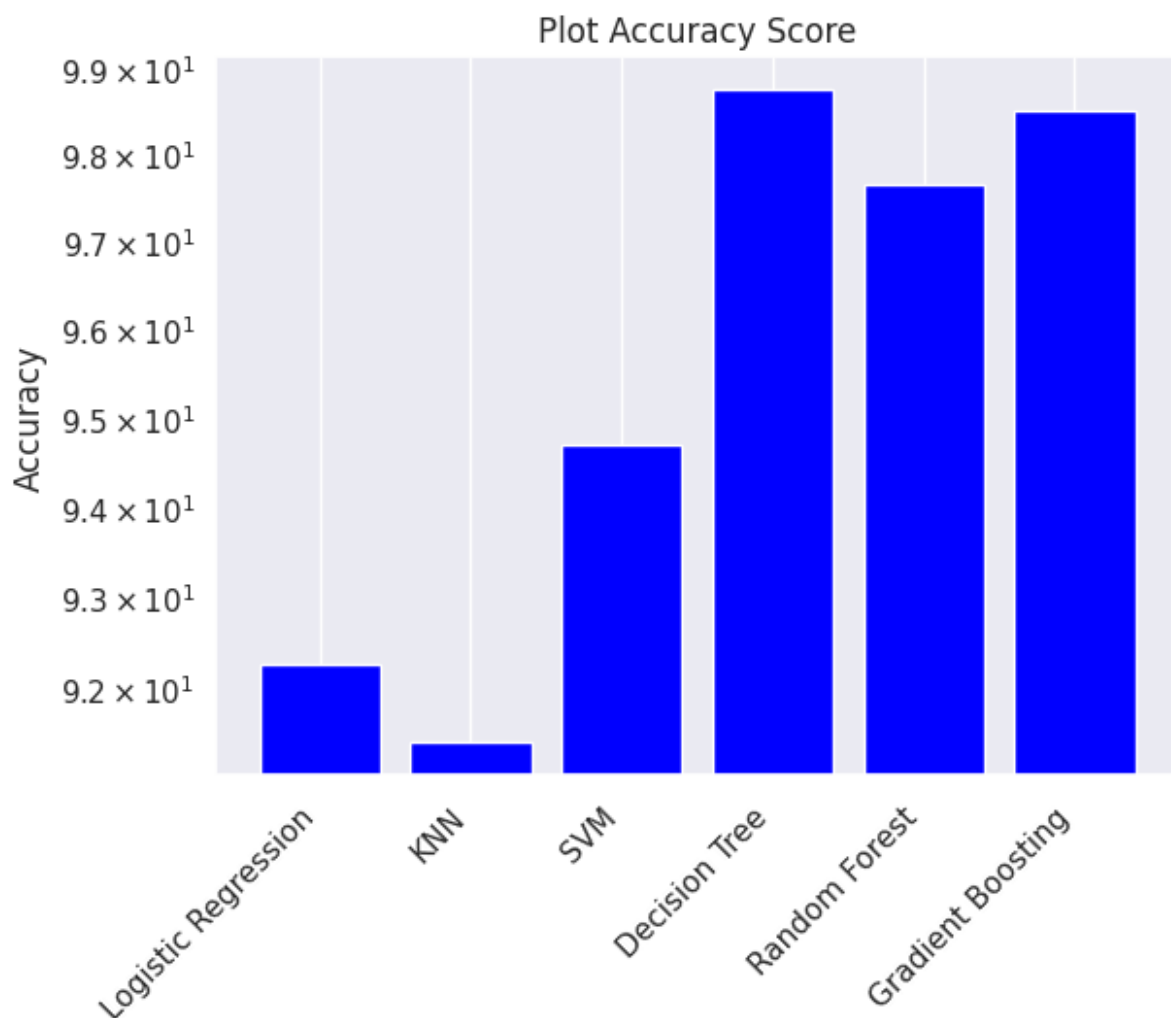
Model	Time
Gradient Boosting	419.636678
SVM	146.919357
Random Forest	66.885382
Logistic Regression	8.994157
KNN	2.406981
Decision Tree	0.323903

Table 5: Time Comparison

7.6 Accuracy Score plotting

Code:

```
1 Model = ['Logistic Regression', 'KNN', 'SVM', 'Decision Tree', 'Random  
Forest', 'Gradient Boosting']  
2 Accuracy = [LRAcc*100, KNAcc*100, SVCAcc*100, DTAcc*100, RFAcc*100,  
GBAcc*100]  
3 fig, plot= plt.subplots()  
4 plot.bar(range(len(Model)), Accuracy, color='blue')  
5 plot.set_yscale('log')  
6 plot.set_xticks(range(len(Model)))  
7 plot.set_xticklabels(Model, rotation=45, ha='right') # Set the xtick  
labels  
8 plot.set_ylabel('Accuracy')  
9 plot.set_title('Plot Accuracy Score')  
10 plt.show()
```



Accuracy Score Plot

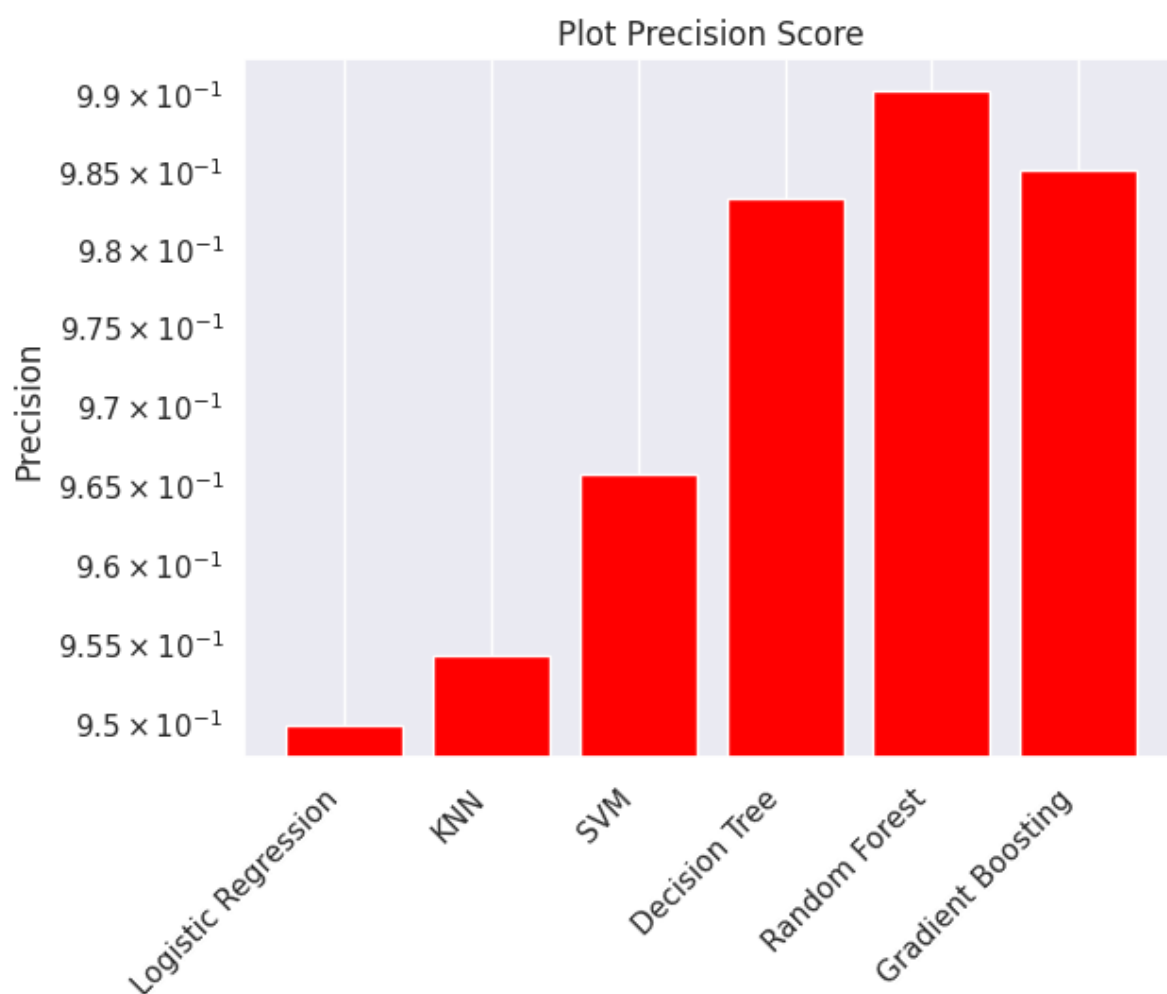
7.7 Precision Score plotting

Code:

```

1 Model = ['Logistic Regression', 'KNN', 'SVM', 'Decision Tree', 'Random
  Forest', 'Gradient Boosting']
2 Precision = [precision_LR, precision_KNN, precision_SVM, precision_DT,
  precision_RF, precision_GB]
3 # Plot Precision Score
4 fig, plot= plt.subplots()
5 plot.bar(range(len(Model)), Precision, color='red')
6 plot.set_yscale('log')
7 plot.set_xticks(range(len(Model)))
8 plot.set_xticklabels(Model, rotation=45, ha='right') # Set the xtick
  labels
9 plot.set_ylabel('Precision')
10 plot.set_title('Plot Precision Score')
11 plt.show()

```



Precision Score Plot

7.8 Recall Score plotting

Code:

```

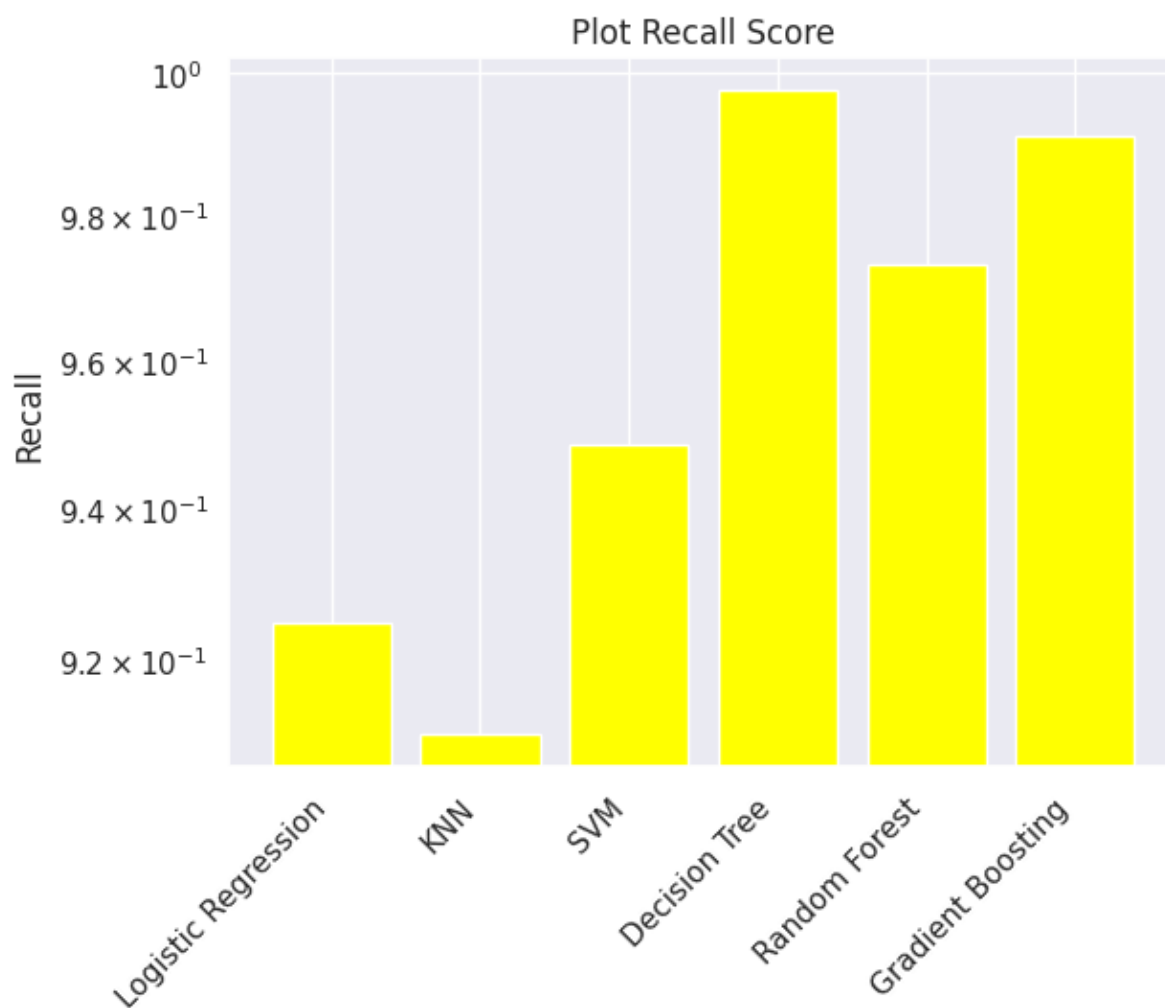
1 Model = ['Logistic Regression', 'KNN', 'SVM', 'Decision Tree', 'Random
  Forest', 'Gradient Boosting']

```

```

2 Recall = [recall_LR, recall_KNN, recall_SVM, recall_DT, recall_RF,
            recall_GB]
3 # Plot Recall Score
4 fig, plot= plt.subplots()
5 plot.bar(range(Model)), Recall, color='yellow')
6 plot.set_yscale('log')
7 plot.set_xticks(range(len(Model)))
8 plot.set_xticklabels(Model, rotation=45, ha='right') # Set the xtick
           labels
9 plot.set_ylabel('Recall')
10 plot.set_title('Plot Recall Score')
11 plt.show()

```



Recall Score Plot

7.9 F1_Score plotting

Code:

```

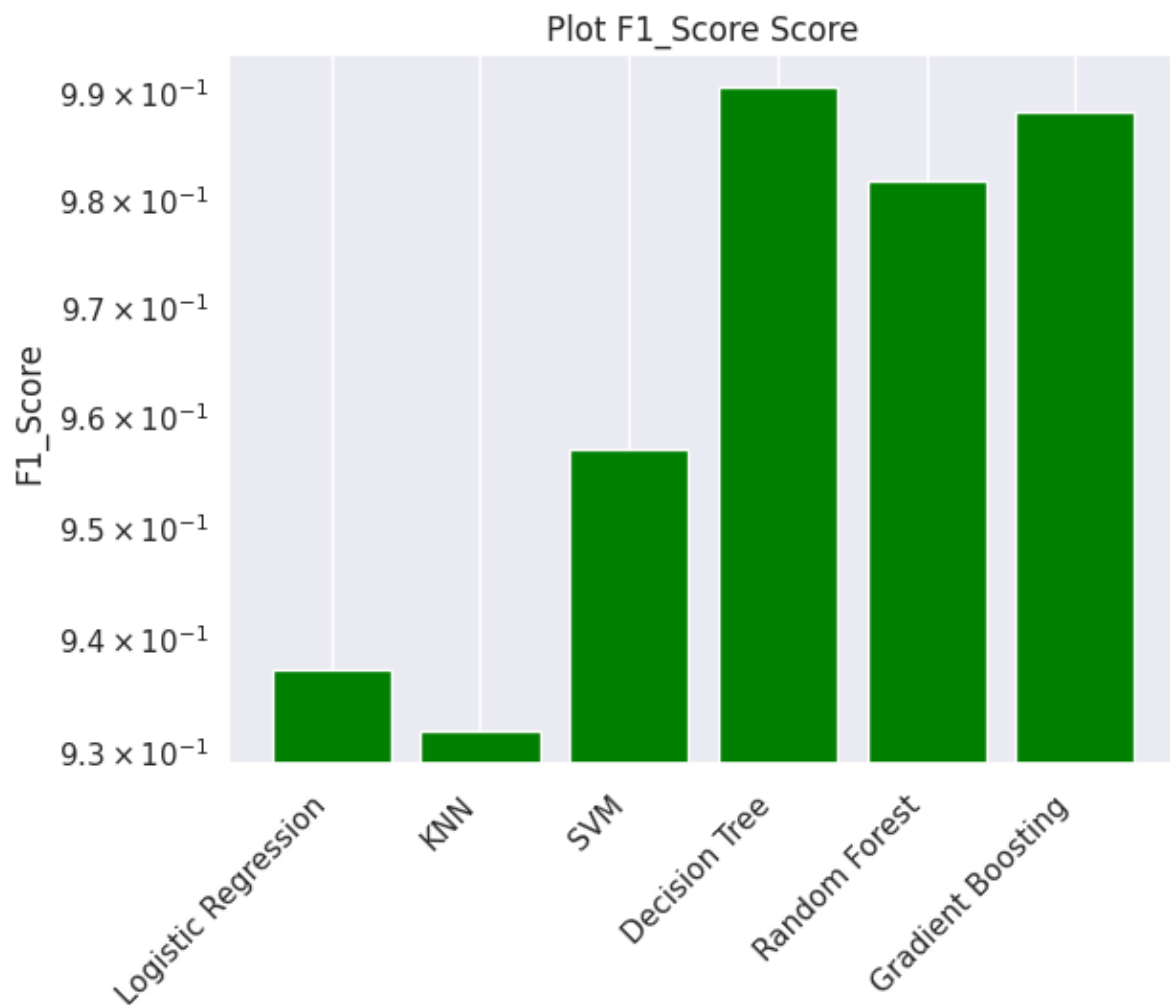
1 Model = ['Logistic Regression', 'KNN', 'SVM', 'Decision Tree', 'Random
           Forest', 'Gradient Boosting']
2 F1_Score = [f1_LR, f1_KNN, f1_SVM, f1_DT, f1_RF, f1_GB]
3
4 fig, plot= plt.subplots()

```

```

5 plot.bar(range(len(Model)), F1_Score, color='green')
6 plot.set_yscale('log')
7 plot.set_xticks(range(len(Model)))
8 plot.set_xticklabels(Model, rotation=45, ha='right') # Set the xtick
   labels
9 plot.set_ylabel('F1_Score')
10 plot.set_title('Plot F1_Score Score')
11 plt.show()

```



F1_Score Plot

7.10 Computational Time Plot

Code:

```

1 Model = ['Logistic Regression', 'KNN', 'SVM', 'Decision Tree', 'Random
   Forest', 'Gradient Boosting']
2 Time = [train_time_LR, train_time_KNN, train_time_SVM, train_time_DT,
   train_time_RF, train_time_GB ]
3 # Computational Time Plot
4 fig, plot= plt.subplots()
5 plot.bar(range(len(Model)), Time, color='purple')
6 plot.set_yscale('log')
7 plot.set_xticks(range(len(Model)))

```

```

8 plot.set_xticklabels(Model, rotation=45, ha='right') # Set the xtick
   labels
9 plot.set_ylabel('Training Time Comparison')
10 plot.set_title('Training Time Comparison')
11 plt.show()

```



Computational Time Plot

7.11 Plot confusion matrix for each model

Code:

```

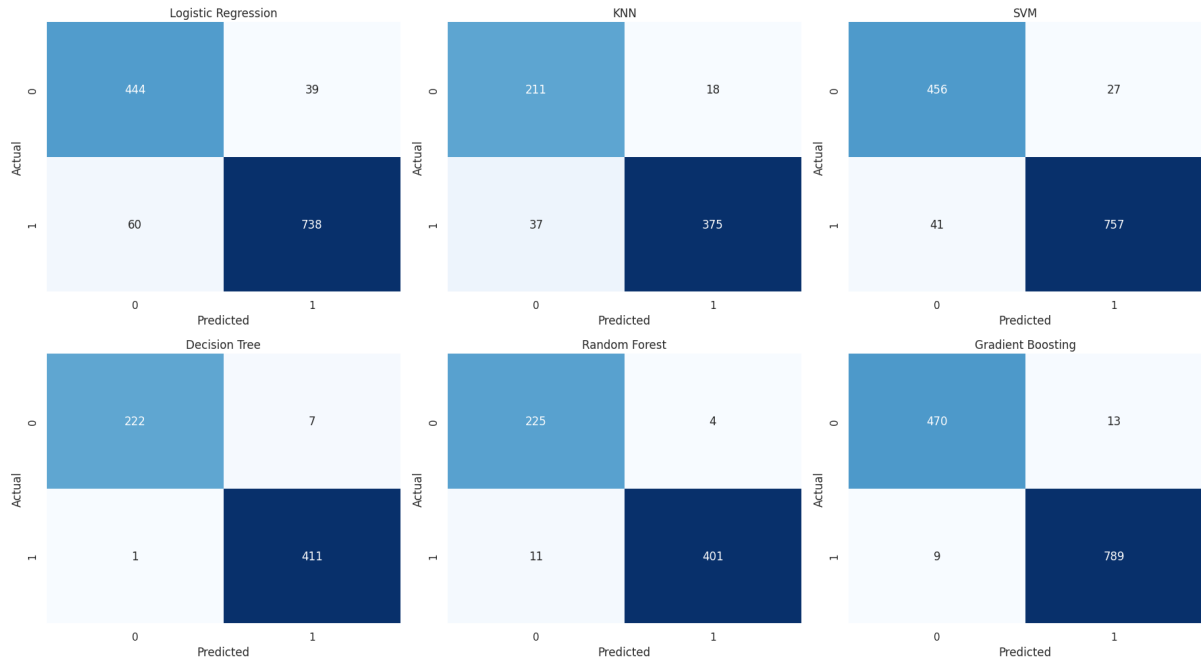
1 Model = ['Logistic Regression', 'KNN', 'SVM', 'Decision Tree', 'Random
   Forest', 'Gradient Boosting']
2 CM = [CM_LR, CM_KNN, CM_SVM, CM_DT, CM_RF, CM_GB]
3 # Create a figure with 6 subplots
4 fig, axes = plt.subplots(2, 3, figsize=(18, 10))
5 axes = axes.ravel()
6
7 # Plot confusion matrix for each model
8 for i, ax in enumerate(axes):

```

```

9     sns.heatmap(CM[i], annot=True, fmt='d', cmap='Blues', cbar=False,
10                ax=ax)
11     ax.set_title(Model[i])
12     ax.set_xlabel('Predicted')
13     ax.set_ylabel('Actual')
14 # Adjust layout
15 plt.tight_layout()
16 plt.show()

```



Confusion Matrix of each model

7.12 Evalutation Matrics:

The following table presents the evaluation metrics for each model, including Accuracy, Precision, Recall, F1-Score, and the corresponding time taken for training.

Model	Accuracy (%)	Precision	Recall	F1-Score	Time (s)
1. Gradient Boosting	98.52	0.9851	0.9912	0.9881	419.64
2. SVM	94.69	0.9656	0.9486	0.9570	146.92
3. Random Forest	97.66	0.9901	0.9733	0.9816	66.89
4. Logistic Regression	92.27	0.9498	0.9248	0.9371	8.99
5. KNN	91.42	0.9542	0.9102	0.9317	2.41
6. Decision Tree	98.75	0.9833	0.9976	0.9904	0.32

Table 6: Model Evaluation Metrics

• Comparison of Model Performance:

The **Decision Tree** and **Gradient Boosting** models achieved the highest accuracy (98.75% and 98.52%, respectively), with **Gradient Boosting** having the best F1-score (0.9881). **KNN** showed the lowest accuracy (91.42%) but was faster than the other models. **SVM** and **Random Forest** performed well with moderate accuracy and training time.

8. Discussion

Based on the results from various models, we can interpret the following key findings:

- **Model Performance:**

- ◆ The **Decision Tree** model achieved the highest **Accuracy** of **98.75%** with a very high **Recall** of **99.76%** and **F1-Score** of **0.99**. This indicates that the Decision Tree model is very effective in identifying both positive and negative instances in the loan approval prediction task.
- ◆ **Gradient Boosting** also performed well, with an accuracy of **98.52%** and a very strong **F1-Score** of **0.99**. The model's high **Precision** (0.99) and **Recall** (0.99) suggest that it can reliably predict the loan approval status without significant misclassification.
- ◆ **Random Forest** and **SVM** both showed good accuracy (around **97%** and **94%**, respectively), with relatively balanced Precision and Recall. However, **SVM** showed a slight drop in performance compared to Random Forest and Gradient Boosting.
- ◆ **Logistic Regression** and **KNN** showed relatively lower performance, with **KNN** performing the worst in terms of accuracy and F1-Score. This could be due to KNN's sensitivity to the feature space and its inability to handle imbalances or noisy data as effectively as the other models.

- **Anomalies/Unexpected Findings:**

- ◆ The **SVM** model, despite having good **Precision**, displayed a lower **Recall** compared to models like **Gradient Boosting** and **Decision Tree**. This suggests that SVM might be more conservative in predicting positive instances, which could result in some false negatives.
- ◆ **KNN**'s relatively low performance may be attributed to its lack of robustness against the curse of dimensionality and its reliance on local neighbors, which may not capture the global trends in the data well.

- **Limitations of the Study:**

- ◆ The models are evaluated using **Accuracy**, **Precision**, **Recall**, and **F1-Score**; however, they might not fully account for the specific business constraints or further operational challenges, such as model interpretability and scalability.
- ◆ **Model Interpretability:** While models like **Decision Trees** and **Logistic Regression** are easier to interpret, models like **Gradient Boosting** and **Random Forest** are considered black-box models, making it difficult to explain the decision-making process behind predictions.

- ◆ **Data Imbalance:** Although techniques like **GridSearchCV** and **RandomizedSearchCV** were used for model selection, the dataset might still contain imbalances that could affect model performance on underrepresented classes. Further balancing techniques, such as oversampling or undersampling, could improve the results.
- ◆ **Computational Complexity:** Models like **Gradient Boosting** and **Random Forest** require more computational resources and time compared to simpler models like **Logistic Regression**, which might be a limitation when scaling the model for large datasets.

9. Conclusion

9.1 Key Takeaways

In this project, we evaluated multiple machine learning models for the task of loan approval prediction, including Decision Tree, Random Forest, Gradient Boosting, KNN, SVM, and Logistic Regression. Among these, **Decision Tree** and **Gradient Boosting** achieved the highest accuracy, demonstrating their effectiveness for this classification task.

The models were evaluated based on multiple metrics, including accuracy, training time, and confusion matrix, which helped in assessing their overall performance and efficiency. Hyperparameter tuning techniques such as **GridSearchCV** and **RandomizedSearchCV** were employed to fine-tune model parameters, which improved model accuracy. Additionally, preprocessing techniques such as feature encoding and normalization were used to optimize model performance.

9.2 Best Model for Loan Eligibility Prediction:

- The **Decision Tree** model stands out as the best model for loan eligibility prediction based on its combination of high accuracy (**98.75%**), excellent recall (**0.9975**), and strong F1 score (**0.990**). This model is highly efficient at detecting eligible applicants without missing many.
- **Gradient Boosting** also performs admirably and can be considered a good alternative to Decision Tree if needed.

9.3 Reflection on Objectives

The primary objective of this project was to predict loan approval status based on available features. This objective was successfully met, as the models provided strong predictions with high accuracy.

Throughout the project, data was properly cleaned, transformed, and split for training, validation, and testing. This ensured reliable model evaluation and comparison. The models were tuned to enhance their performance, and the results were compared to select the best-performing model.

9.4 Suggestions for Future Research or Improvement

- **Model Selection:** While Decision Tree and Gradient Boosting performed well, further exploration of models such as XGBoost, LightGBM, or deep learning approaches could yield even better results.
- **Feature Engineering:** Additional feature engineering could be performed to extract more meaningful features from the data, which might improve model accuracy. For example, incorporating external data like economic factors could enhance prediction performance.
- **Handling Class Imbalance:** Although class imbalance was not directly addressed in this project, future work could involve techniques like SMOTE or adjusting the decision threshold to handle imbalance better.
- **Model Deployment:** Future work could focus on deploying the best-performing model into a production environment and monitoring its real-world performance to ensure continued accuracy.
- **Evaluation Metrics:** Future research could include a more extensive analysis of model evaluation metrics such as ROC-AUC and precision-recall curves to better understand performance in imbalanced datasets.

10. References

- **ChatGPT:** Used for providing support and suggestions related to project tasks, model selection, and hyperparameter tuning.
 - **'Hands-On Machine Learning' by Aurélien Géron:** A valuable resource for machine learning concepts and practical implementation. This book was used to understand the fundamentals of algorithms and techniques used in the project.
 - **Google:** A search engine for gathering datasets, tutorials, and general information related to the project.
 - **GitHub:** An online platform for hosting and sharing code. Used for accessing machine learning libraries, example codes, and repositories relevant to the project.
 - **Kaggle:** An online platform for data science and machine learning competitions, used to collect datasets relevant to the project.
-